Duplicate and Near-Duplicate File Finder

A disk cleaner built on hashing, BK-trees, and union-find | CSC 212 Final Project Proposal, Fall 2026

1. Team and Repository

Team members	[Name 1], [Name 2], [Name 3]
Discussed with	[TA / instructor name] on [date]
GitHub repository	[https://github.com/<user>/<repo>]

2. Problem and Motivation

Everyone's computer slowly fills up with copies: the same lecture PDF downloaded three times, project folders duplicated "just in case", and photo bursts from a phone where a dozen shots are nearly identical. Existing tools exist, but students rarely know how they work or why some are fast and others freeze on a large disk. We want a tool we will actually run on our own machines that finds **exact duplicates** and **near-duplicate photos** (the same picture resized, recompressed, or slightly cropped) and reports how much space can be reclaimed.

The obvious approach compares every file with every other file byte by byte. With *n* files this is $O(n^2)$ comparisons, and each one may read whole files from disk: for 100,000 files that is roughly five billion pairs. Near-duplicate photos are harder still, because identical bytes are not required. Our goal is to show that the right data structures turn this into something that finishes in seconds, and to measure exactly how much work each structure saves.

3. Data Structures and Why They Fit

The structures below will be implemented by the team in C++ (no STL containers for the core).

Structure	Role in the project	Why it beats the obvious alternative
Hash table (custom, open addressing)	Groups files by size, then by partial hash (first block), then by full-content hash.	Files that cannot be equal are discarded without ever being compared; most files are never fully read. O(1) expected grouping vs. $O(n^2)$ pairwise comparison.
BK-tree (metric tree on Hamming distance)	Stores 64-bit perceptual hashes of images and answers "find all hashes within distance d of this one".	Uses the triangle inequality to skip entire subtrees, so a query compares against a small fraction of the stored hashes instead of all of them.
Union-find (path compression, union by rank)	Merges files into duplicate groups as matches are discovered (A matches B, B matches C: one group).	Near-constant time per merge and query; avoids rebuilding groups or searching graphs of matches.
Binary heap (priority queue)	Surfaces the groups that would reclaim the most space first.	O(log n) extraction of the top-k groups, instead of sorting everything.

How it fits together. *Exact duplicates:* group by size, then by hash of the first 4 KB, then by hash of the full file, and finally confirm with a byte comparison. *Near-duplicate photos:* compute a 64-bit perceptual hash per image (a fingerprint that changes little when the picture changes little), insert all hashes into the BK-tree, query each one with a small distance threshold, and merge the matches with union-find.

Supporting code (libraries allowed): file-system traversal (std::filesystem), image decoding (e.g., stb_image), and a standard hash function for file contents. The core structures above are ours.

4. Baseline and How We Will Measure

Baselines (one per component, so each structure is justified separately):

- **Exact duplicates:** compare every pair of files byte by byte; and a middle baseline that fully hashes every file with no size grouping.
- **Near-duplicates:** compare every pair of perceptual hashes with a linear scan, vs. the BK-tree.
- **Grouping:** repeatedly relabeling a group array on each match, vs. union-find.
- **Top-k report:** sort all groups by reclaimable space, vs. the heap.

Experiments (plots of time vs. input size, as in Lab 1):

- Total time vs. number of files (1,000 to 200,000) on generated datasets, for each baseline and for our pipeline.
- Bytes actually read from disk at each stage of the exact-duplicate pipeline (after size grouping, after partial hash, after full hash).
- Number of distance computations per query: BK-tree vs. linear scan, for thresholds from 0 to 12 bits. We expect the BK-tree advantage to shrink as the threshold grows, and we will report where it stops paying off.
- Quality: precision and recall of near-duplicate detection vs. Hamming threshold, using datasets with known ground truth.
- Memory used by each structure.

Datasets. Benchmarks use generated data so they are reproducible: synthetic files with planted duplicates, and an image set (from public-domain or freely licensed photos) in which a script creates known variants (resized, recompressed, brightness-shifted, lightly cropped). Because we know which files are copies, we can compute exact precision and recall. The live demo additionally runs on the team's own folders.

Correctness. Unit tests for each structure on its own (hash table insert/lookup/collisions, BK-tree results compared with a linear scan on random data, union-find invariants, heap order). Integration tests on small directories with a known answer. Every exact-duplicate group is verified byte for byte before it is reported.

5. What the Finished Project Looks Like

Command-line tool. A single command scans one or more folders and prints a summary such as "3,412 duplicate files in 1,208 groups; 14.2 GB can be reclaimed", with the largest groups first. It also reports how long each stage took and how many comparisons it avoided.

HTML report. The engine writes a results file and a lightweight web page displays it: duplicate groups with file paths and sizes, and for photos, the near-duplicates shown side by side with their distance score and a slider that changes the similarity threshold. The report lets the user decide what to keep.

Showcase demo. We will:

- run the tool live on a real folder from one of our own computers and show the reclaimable space;

- toggle between the naive all-pairs method and our pipeline on the same dataset, with a live timer and comparison counter;
- show the benchmark plots and the precision/recall curve for the photo matching.

Safety by design. The tool is read-only by default (report only). Any cleanup is opt-in, never deletes automatically, and at most moves files to a quarantine folder that the user can restore from.

6. Scope and Timeline

Phase	Deliverables
Core (committed)	Directory scanner, custom hash table, exact-duplicate pipeline (size, partial hash, full hash, byte verification), union-find grouping, baselines, test data generator, benchmarks, report.
Extensions (planned)	Perceptual hashing of images, BK-tree with linear-scan baseline, precision/recall evaluation, heap-based top-k ranking, HTML side-by-side viewer.
Stretch (if time allows)	Alternative index for near-duplicates (e.g., VP-tree or multi-index hashing) and comparison with the BK-tree; multi-threaded hashing.

Weeks	Milestone
1-2	Repository, scanner, custom hash table, size grouping, partial and full hashing, first tests, data generator.
3-4	Byte verification, union-find, naive baselines, first benchmark plots for exact duplicates.
5-6	Perceptual hash, BK-tree and linear-scan baseline, image variant generator, precision/recall experiments.
7	Heap-based ranking, command-line report, threshold tuning, HTML viewer.
8	Polish, final benchmarks on the team's own folders, README, REPORT.md, showcase rehearsal.

We will commit weekly from every team member. Each member will own at least one structure and be able to explain its code at the showcase. Proposed split: [Member 1: hash table, exact-duplicate pipeline, scanner; Member 2: perceptual hashing, BK-tree, quality experiments; Member 3: union-find, heap, benchmarks, report viewer].

7. Risks and Mitigation

- **False positives in photo matching:** perceptual hashes can group images that merely look alike. Mitigation: a conservative default threshold, side-by-side display, and never acting automatically.
- **BK-tree slows down at large thresholds:** the metric pruning weakens as the allowed distance grows. Mitigation: measure it honestly, keep thresholds small, and treat an alternative index as a stretch comparison.
- **File-system edge cases:** hard links, symbolic links, unreadable files, and very large files. Mitigation: detect and skip links, count hard links once so reclaimable space is not overstated, and log errors without stopping.
- **Safety and privacy:** the tool handles personal files. Mitigation: everything runs locally with no network access, the default mode is read-only, and no file is deleted without an explicit user action.